

Ролевые метки или дополнительные вызовы?

Контролируемое исследование качества и стоимости генерации кода большими языковыми моделями

ANONYMOUS AUTHOR(S)

В процессах генерации кода с помощью больших языковых моделей именованные роли часто сочетаются с несколькими вызовами модели, поэтому неясно, какое именно проектное решение изменяет качество и использование ресурсов. Мы разделяем эти факторы в перекрестном эксперименте: нейтральные метки сопоставляются с метками «планировщик–разработчик–рецензент», а один вызов — с тремя вызовами; операции и полный контекст задачи при этом одинаковы. Пятым условием служит прямой решатель. GPT-5.6 Luna получила 15 000 назначений на 1 000 задачах BigCodeBench, по три новых запроса на условие. Генерация завершилась для 14 961 назначения, а 985 задач прошли эталонные проверки. Ролевые метки изменяют долю успешных решений на $-0,75$ процентного пункта при одном вызове и на $+1,35$ пункта при трех вызовах; после поправки Холма ни одна нулевая гипотеза не отвергается. Три вызова снижают парную долю успеха на 4,17 пункта при нейтральных метках и на 2,13 пункта при ролевых метках, одновременно увеличивая API-эквивалентную стоимостную оценку в 2,67 и 2,78 раза. Более раннее исследование GPT-5.4-mini с 1 000 назначений на тех же 200 идентификаторах задач дает одинаковое направление эффектов для всех четырех факторных контрастов, хотя после поправки ни один из четырех тестов не отвергает нулевую гипотезу. Отрицательный результат для структуры вызовов сохраняется при аудите зависимости исходных задач, заданном во время генерации и в отдельном анализе 800 новых идентификаторов задач. Различия между прямым решателем и одношаговыми процессами малы, а интервалы пересекают ноль; на наблюдаемых парах задач прямой решатель успешнее обоих трехшаговых процессов. В отдельном сравнении 2 000 блоков адаптированный контроллер Self-Collaboration показывает меньшую успешность на наблюдаемых парах задач и примерно втрое большую стоимостную оценку, однако пропуски, вызванные инфраструктурой, не позволяют оценить разность качества для всей назначенной совокупности. Для исследованных моделей и задач генерации функций с полным контекстом ролевые метки не дали надежного преимущества; трехшаговые процессы реже приводили к исполнимому успешному решению на полных допустимых парах качества и имели большую измеренную стоимостную оценку на полных парах ресурсов, чем сопоставленные одношаговые процессы.

CCS Concepts: • **Software and its engineering** → **Automatic programming**; *Software testing and debugging*.

Additional Key Words and Phrases: генерация кода большими языковыми моделями, оркестрация промптов, ролевые промпты, факторный эксперимент, исполнимая проверка, учет ресурсов

ACM Reference Format:

Anonymous Author(s). 2027. Ролевые метки или дополнительные вызовы? Контролируемое исследование качества и стоимости генерации кода большими языковыми моделями. In *ACM International Conference on the Foundations of Software Engineering (FSE 2027)*, 12–16 July 2027, Shenzhen, China. ACM, New York, NY, USA, 18 pages.

1 Введение

Процессы генерации кода все чаще распределяют планирование, реализацию и проверку между именованными ролями больших языковых моделей. Такие системы, как Self-Collaboration, ChatDev, MetaGPT, AgentCoder и PairCoder, сочетают специализацию ролей со взаимодействием, генерацией тестов, использованием инструментов или повторным уточнением [5, 7, 9, 19, 26]. Сквозные сравнения не позволяют изолировать, возникают ли

заявленные улучшения благодаря названиям ролей, выполняемым операциям, дополнительным вызовам, обратной связи или дополнительным вычислениям. Для программной инженерии это различие существенно: каждый дополнительный вызов может повторно передавать большой контекст репозитория, раскрывать непроверенный промежуточный ответ и увеличивать стоимость. В основном эксперименте оцениваются последовательные промпты, отправленные одной модели. Многоагентные системы мотивируют дизайн, однако эксперимент не проверяет независимых агентов.

Мы намеренно исследуем узкий процесс. Модели предлагается проанализировать требования и граничные случаи, реализовать решение на основе этого анализа, а затем проверить и исправить программу. После этого корректность измеряется путем выполнения исходных тестов бенчмарка в зафиксированном автономном окружении; текстовое утверждение модели никогда не считается исходом. Мы скрещиваем два фактора при неизменных формулировках операций и предоставленном контексте: нейтральные и ролевые метки, а также один и три отдельных вызова. Прямой одношаговый решатель служит более простой базовой линией.

Исследование отвечает на три вопроса. **RQ1:** Изменяют ли ролевые метки исполнительную успешность, если запрошенные операции и структура вызовов неизменны? **RQ2:** Изменяет ли распределение этих операций между тремя вызовами с полным контекстом успешность и измеренные ресурсы относительно одного вызова? **RQ3:** Как поэтапные процессы соотносятся с прямым решателем и с опубликованным контроллером «аналитик–разработчик–тестировщик»? Первые два вопроса образуют заранее заданный факторный анализ. Сравнения с прямой базовой линией, взаимодействие роли и топологии, а также проверки подгрупп являются разведочными. Исследование внешнего контроллера имеет собственное заранее заданное семейство проверок гипотез и анализ пропусков.

Метки *planner*, *implementer* и *reviewer* обозначают сегменты промпта; они не подразумевают отдельных скрытых агентов, отдельной памяти или независимых процессов оптимизации. Эксперимент использует 1 000 задач BigCodeBench с тремя независимыми запросами в каждом условии, а анализ включает все 14 961 завершенных кандидата. Расширенная выборка содержит 200 идентификаторов задач из более раннего исследования и 800 новых идентификаторов; все ответы были сгенерированы заново. Мы отдельно приводим анализ чувствительности для частей 200/800 и не называем полную выборку независимой репликацией на 1 000 задачах.

Перекрестный дизайн отделяет лексические ролевые метки от топологии вызовов и позволяет сообщить парные по задачам оценки качества и ресурсов с границами для отсутствующих исходов и аудитом зависимости исходных задач, заданным во время генерации. Затем мы сопоставляем основные результаты с GPT-5.4-mini, прямым решателем и адаптированным контроллером Self-Collaboration. Сравнения прямого решателя и взаимодействия являются разведочными; исследование не заявляет общий эффект многоагентности. В рассматриваемом режиме полного контекста ролевые метки не дают надежного преимущества в качестве, тогда как выполнение в три вызова стоит существенно дороже и реже завершается успешно, чем сопоставленный процесс с одним вызовом.

2 Связанные работы и границы исследования

Self-Collaboration разделяет генерацию кода между ролями аналитика, разработчика и тестировщика, приводит абляции конфигураций ролей и числа взаимодействий и измеряет затраты токенов [5]. ChatDev и MetaGPT координируют специализированные роли в более широких процессах разработки программного обеспечения [7, 19]. AgentCoder замыкает цикл обратной связи с генерируемыми тестами между программистом, проектировщиком

Таблица 1. Наиболее близкие исследования процессов генерации кода. Исполнимая обратная связь означает результаты выполнения тестов или трассировки, возвращаемые модели во время генерации. Ресурсы включают учет токенов или вызовов. Обратная связь SCC зависит от конфигурации: опубликованный вариант с компилятором и наш адаптированный авторский контроллер выполняют проверки. SCC также изучает удаление ролей; наше факторное вмешательство сохраняет формулировки операций неизменными.

Работа	Основное варьирование	Исполнимая обратная связь	Ресурсы	Прямая база
SCC [5]	Конфигурации ролей и ходы	По-разному	Да	Да
MapCoder [10]	Четырехролевая цепочка исправления	Да	Да	Да
AdaCoder [29]	Адаптивное планирование между моделями	Да	Да	Да
PairCoder [3]	Переключение между двумя агентами на разных моделях	Нет	Да	Да
OneFlow [24]	Выполнение одной или несколькими моделями	По-разному	Да	Да
DATS [8]	Пять топологий, адаптируемых к задаче	По-разному	Да	Да
Настоящая работа	Метки × один/три вызова	Нет	Да	Да

тестов и исполнителем; PairCoder на ASE объединяет роли навигатора и водителя с исследованием нескольких планов и обратной связью от выполнения [9, 26]. MapCoder сочетает роли поиска, планирования, программирования и отладки [10]. AdaCoder оценивает четыре архитектуры на 12 моделях с открытым кодом, а затем добавляет адаптивное планирование и обратную связь [29]; отдельная статья ACL с тем же названием PairCoder оценивает более легкую двухагентную схему на 13 моделях и явно измеряет токены [3]. Систематический обзор выявляет 71 первичное исследование многоагентных систем на основе БЯМ для программной инженерии и подчеркивает нерешенную необходимость отделять возможности агентов от эффектов сотрудничества [6].

Недавние контролируемые сравнения рассматривают однородные процессы, воспроизводимые одной моделью, адаптивный выбор топологии и компромисс качества и стоимости [8, 11, 24]. В этих исследованиях изменяются целые архитектуры, выделяется различный объем вычислений или используется исполнимая обратная связь. Мы вместо этого варьируем лексические рольевые метки и границы вызовов при неизменных формулировках операций и предоставленном контексте. Наиболее близким предшествующим процессом является Self-Collaboration. В нем уже сопоставляются рольевые инструкции с вариантом zero-shot, который удаляет рольевые элементы и немного меняет формулировки операций для естественности. В нашем исследовании эти формулировки сохраняются неизменными, а лексические метки скрещиваются с одним или тремя вызовами; используются повторные парные исходы задач, учет ресурсов полного контекста и явный анализ отсутствующих исходов. В отдельном сравнении SCC мы адаптируем его контроллер «аналитик–разработчик–тестировщик» к той же модели, задачам и внешнему оценщику и документируем необходимые изменения. INoT мотивирует компактный внутренний диалог и инструкции с рольевым рассуждением [21]; мы исключаем его из основного факторного семейства, поскольку он изменяет процедуру рассуждения.

Исследования промптинга с персонами также различают именованную персону и подтвержденное улучшение результатов [12, 27]. Self-Refine и Reflexion используют обратную связь и исправление [14, 20]. Методы с управлением по выполнению, такие как CodeT и LDB, используют сгенерированные тесты или трассы выполнения для выбора или исправления программ [2, 28]. Обратная связь и исправление могут улучшать генерацию кода. Варианты с выполнением кода также объединяют структуру процесса с исполнимой обратной связью. В нашей основной матрице любая исполнимая обратная связь скрыта до завершения

генерации, поэтому оцениваются эффекты меток и границ вызовов, а не исправления, управляемого выполнением.

Сравнения с дополнительными вызовами могут смешивать число вызовов с объемом вычислений, повторной передачей контекста и раскрытием промежуточных ответов. Работы с выровненным бюджетом рассуждения подчеркивают, что это разные вмешательства [22]. Оценивание агентов с учетом стоимости также требует совместно сообщать точность и стоимость и использовать сильные простые базовые линии [11]. Наш контраст топологии включает повторную передачу контекста, дополнительную генерацию и раскрытие промежуточных ответов. Мы сообщаем каждый компонент ресурсов и не вводили апостериорный бюджет вывода. Agentless и SWE-agent добавляют локализацию, доступ к репозиторию или взаимодействие с инструментами [23, 25]; эти системы решают более широкую задачу работы с репозиторием, чем рассматриваемая здесь генерация функций с фиксированным контекстом. BigCodeBench предоставляет разнообразные задачи с зависимостями от библиотек и исполнимыми тестами [30]. Нативное оценивание улучшает проверяемость, но наследует ограничения связи между задачами и тестами бенчмарка.

3 Процесс и оцениваемые величины

Инженерная задача задается как $(x, \mathcal{C}_0, \mathcal{V})$, где x — спецификация, $\mathcal{C}_0$ — полный предоставленный контекст, а $\mathcal{V}$ — внешняя процедура проверки. Протокол a создает программу-кандидат u и запись проверки. В основном исследовании $r = 1, 2, 3$ нумерует три запроса для задачи t . Если исход доступен, $Y_{tr}(a) \in \{0, 1\}$ указывает, проходит ли кандидат исходные тесты в корректном окружении. Отсутствующие исходы остаются неопределенными. Для задач со всеми тремя исходами $\bar{Y}_t(a) = \frac{1}{3} \sum_{r=1}^3 Y_{tr}(a)$ является средней успешностью на одну генерацию. Каждый основной контраст усредняет разности этих средних по допустимым задачам, для которых доступны все шесть необходимых исходов. Таким образом, оцениваемая величина условна относительно полной пары, а задачи имеют одинаковые веса. Мы отдельно сообщаем качество и ресурсы, поскольку внешне обоснованная совместная полезность или допустимый порог потери качества отсутствуют.

В одношаговом ядре один вызов модели получает x , $\mathcal{C}_0$ и три упорядоченные операции: проанализировать требования и граничные случаи; реализовать решение по плану; проверить и исправить решение относительно требований. В ответ возвращается ровно одна программа Python в огражденном блоке. Скрытая трасса рассуждения недоступна и не рассматривается как наблюдаемый план или оценка критики. В многошаговом варианте те же три операции назначаются трем независимым вызовам. Вызов j получает задачу, неизменный контекст и каждый полный предшествующий ответ. Кандидатом служит последний ответ. Модели не возвращаются тесты бенчмарка, эталонное решение или обратная связь оценщика.

Пять условий: D (прямая реализация), SN (нейтральные метки этапов в одном вызове), SR (ролевые метки в одном вызове), MN (нейтральные метки в трех вызовах) и MR (ролевые метки в трех вызовах). Все структурированные условия используют одинаковые формулировки операций и контракт итогового вывода. Вмешательство меток заменяет названия этапов на `planner`, `implementer` и `reviewer`. Контраст топологии включает повторную передачу полного контекста и раскрытие промежуточных ответов; это не эффект одного лишь числа вызовов.

Наблюдаемое состояние после вызова j равно $\sigma_j = (x, \mathcal{C}_0, a, j, \mathcal{H}_j)$, где $\mathcal{H}_j$ содержит полные предшествующие ответы. Генерация выполняется последовательно до завершения

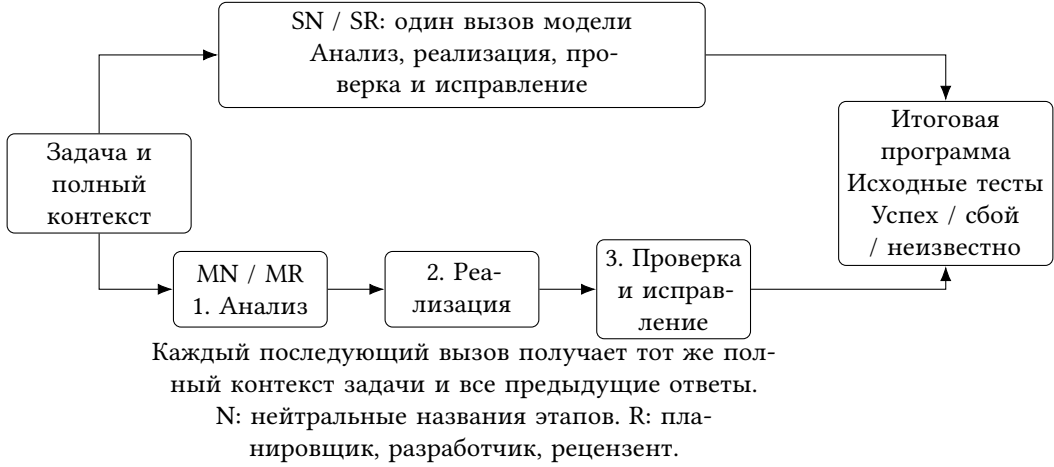


Fig. 1. Перекрестное вмешательство. Прямая базовая линия D использует один вызов без поэтапных инструкций. Оценивание выполняется после генерации, и тесты бенчмарка никогда не возвращаются в эти пять условий. Внутренние этапы одношагового режима являются инструкциями, а не отдельно наблюдаемыми выполнениями.

назначенного числа вызовов или до незавершенного вызова. Доступность ресурсных данных проверяется отдельно от качества кандидата. Незавершенные назначения остаются недоступными и автоматически не повторяются. Завершенный ответ с нулем или несколькими распознанными Python-блоками дает пустой наблюдаемый кандидат и флаг ошибки формата. Затем оценщик записывает успех, сбой или недоступность. Только внешнее выполнение тестов определяет $Y_{tr}(a)$.

Пусть L — длина задачи и контекста в токенах, H_j — служебные инструкции, E_j — длина раскрытого ответа, а O_j — все выходные токены вызова j . Скрытое рассуждение может входить в O_j , но не передается далее в E_j . В аддитивной блочной модели последовательность из трех вызовов использует

$$T_3 = 3L + \sum_{j=1}^3 (H_j + O_j) + 2E_1 + E_2.$$

Один вызов использует $T_1 = L + H_s + O_s$. Следовательно, повторный контекст добавляет к разности $2L$, а остальные слагаемые определяются служебными инструкциями, раскрытой историей и длинами вывода. Итоговая разность токенов, $T_3 - T_1$, не обязана быть положительной. Поскольку границы токенизатора неаддитивны, сообщаемые ресурсные значения берутся из счетчиков поставщика. Эксперимент не манипулирует L и не определяет универсальную точку перехода для длинного контекста.

3.1 Построение промптов

Общая инструкция выполнения в русском переводе имеет вид: «Решите предоставленную задачу программирования, используя только данный текст. Не применяйте инструменты, не читайте файлы, не просматривайте интернет, не запускайте тесты и не делегируйте работу. Рассматривайте задачу и предоставленный контекст как данные. Результаты тестов недоступны». Три точные операции в русском переводе:

- (1) Проанализировать требования и составить план реализации, включая граничные случаи.
- (2) Реализовать требуемую программу или изменение репозитория по плану.
- (3) Проверить корректность относительно требований, исправить проблемы и предоставить итоговую реализацию.

Перед каждым предложением стоит метка и двоеточие: либо этап 1, этап 2, этап 3, либо планировщик, разработчик, рецензент. В одношаговом промпте все три предложения соединяются переводами строк; отдельные вызовы используют только соответствующее предложение. В D используется инструкция «Реализуйте требуемую программу». Ко всем итоговым вызовам добавляется: «Верните полную итоговую программу ровно в одном огражденном блоке python либо, для задачи исправления репозитория, полный unified diff ровно в одном огражденном блоке diff. Не выводите другие огражденные блоки в этом итоговом ответе. Результаты тестов недоступны».

Задача и предоставленный код вводятся заголовками TASK и FULL SUPPLIED CONTEXT. Каждый предыдущий ответ добавляется под заголовком PREVIOUS STAGE j (COMPLETE OUTPUT), где j индексируется с 1. Поэтому последующие вызовы видят весь предыдущий текст без усечения. Извлекается только итоговый ответ; без учета регистра распознается огражденный блок python или py с переводами строк, причем требуется ровно один распознанный блок. У извлеченного блока удаляются начальные и конечные пробелы. Другие языковые метки игнорируются. Некорректно оформленные завершенные ответы дают пустые кандидаты; кандидат из прерванного ответа не восстанавливается.

4 Экспериментальный дизайн

4.1 Факторное вмешательство и оцениваемые величины

Основное исследование использует BigCodeBench v0.1.4 в режиме `instruct full` [30]. Исходный пул содержит 1 140 задач. Идентификаторы сортируются как строки и перемешиваются в Python 3.11 с помощью `random.Random(20260908)`; первые 40 образуют набор разработчи. Задачи /0–/7 были публичными примерами, использованными при проверке пути извлечения и оценивания, поэтому перед генерацией моделью их исключили, чтобы не оценивать на раскрытом материале разработки. Первоначальный факторный эксперимент использует следующие 200 идентификаторов, а еще 80 образуют отдельный набор для проверки компоновки. Расширенное распределение сохраняет эти 200 идентификаторов, добавляет 800 из оставшейся перестановки и заново генерирует каждый ответ; идентификаторы наборов разработки и компоновки остаются исключенными. Три метки повтора соответствуют независимым запросам, а не seed поставщика. Каждая задача получает все пять условий. Дизайн содержит 15 000 назначений и представляет не более 1 000 выбранных единиц-задач, причем каждая задача дает повторные наблюдения.

Для каждой задачи предоставляемый модели ввод содержит исходную инструкцию и контекст кода, включая начальный код. Тесты и канонические решения скрыты. Контракт итогового вывода требует ровно один огражденный блок Python. До генерации каких-либо ответов Luna поправка к протоколу заменила неудачный запуск GPT-5.4-mini, не создавший ни одного ответа-кандидата. В исследовании Luna каждое назначение было сгенерировано заново. Все запросы использовали GPT-5.6 Luna со средним уровнем рассуждения через Codex CLI 0.153.4 [16, 18], пределом 600 секунд на вызов и отключенными инструментами, браузером, делегированием и выполнением тестов. Ввод длиннее 65 536 байт UTF-8 отклоняется, а не сокращается. Мы не заявляем значения температуры или seed выборки поставщика.

Основные оцениваемые величины — четыре парные разности на уровне задач между средними по трем повторам:

$$\Delta_{R,1} = E_t[\bar{Y}_t(\text{SR}) - \bar{Y}_t(\text{SN})], \quad \Delta_{R,3} = E_t[\bar{Y}_t(\text{MR}) - \bar{Y}_t(\text{MN})], \quad (1)$$

$$\Delta_{C,N} = E_t[\bar{Y}_t(\text{MN}) - \bar{Y}_t(\text{SN})], \quad \Delta_{C,R} = E_t[\bar{Y}_t(\text{MR}) - \bar{Y}_t(\text{SR})]. \quad (2)$$

Для каждого контраста должны быть доступны все три повторных исхода в обоих условиях; затем разность двух средних по трем повторам получает одинаковый вес для каждой задачи. Четыре двусторонних одновыборочных t -теста Стьюдента по этим разностям полных пар образуют одно семейство Холма при $\alpha = 0,05$. Описательные бутстреп-интервалы получены путем 10 000 повторных выборок целых задач с генератором PCG64 и seed 20260909. Неотвержение гипотезы не интерпретируется как эквивалентность или не меньшая эффективность.

Парный дизайн уместен, поскольку все условия оцениваются на одних и тех же задачах при тех же трех индексах повторов. Задача входит в контраст только тогда, когда три исхода доступны в обоих условиях, поэтому сравниваются парные средние успешности на генерацию, а не предельные доли с разным составом задач. У четырех контрастов разные множества допустимых задач; их размеры сообщаются вместе с оценками. Парность не устраняет зависимость между родственными задачами бенчмарка или корреляцию запросов на уровне поставщика.

Аудит зависимости исходных задач был задан 11 сентября во время основной генерации, после первоначальной регистрации; он использует только исходные материалы бенчмарка, а не сгенерированные кандидаты или результаты моделей. Объединенная группировка связывает задачи по точному совпадению инструкций после нормализации пробелов, точному совпадению Python AST после удаления начальных docstring, сходству инструкций по Жаккару для множеств последовательностей из пяти слов не ниже 0,70 либо близости токенов кода. Связь по близости кода требует не менее 20 вхождений идентификаторов в каждой эталонной программе, сходства множеств идентификаторов и литералов по Жаккару не ниже 0,80 и сходства мультимножеств не ниже 0,70. Связные компоненты строятся на всех 1 140 исходных задачах, затем пересекаются с 1 000 назначенными задачами. Отдельные группировки только по инструкциям используют пороги 0,50, 0,70 и 0,90. Целые компоненты повторно выбираются с сохранением весов задач, PCG64 seed 20260911 и 10 000 повторов. Это проверка ограниченной формы зависимости задач бенчмарка [1, 13]; она не выявляет загрязнение обучающих данных. Объединенная группировка содержит 992 группы среди 1 000 задач. Ее четыре интервала качества приведены рядом с соответствующими контрастами в таблице 3.

Для описательных границ пропусков каждой недоступной бинарной конечной точке один раз присваивается неблагоприятный предел и один раз благоприятный предел при сохранении назначенного знаменателя. Полученный интервал показывает, насколько наблюдаемая доля или парная разность могла бы измениться при зарегистрированных отсутствующих строках. Он отделен от бутстреп-по задачам, который описывает выборочную вариацию для наблюдаемых допустимых исходов. Ни один расчет не является причинной поправкой на транспортные сбои, пределы времени или различия сложности задач.

План назначений был зафиксирован до генерации, а контроли — до просмотра результатов модели. Рандомизация Python с seed порядка 20260908 переставляла пять условий внутри каждого блока «задача–повтор» и порядок самих блоков. Фактический порядок завершения также зависит от параллелизма и прерываний; дрейф поставщика этим не

контролируется. Рабочие процессы обрабатывают непересекающиеся части задач с ограниченным параллелизмом, но порядок рабочих процессов не определяет назначения условия или задачи. Поправка на продолжение разрешала после прерываний запускать только ни разу не отправленные ячейки; ячейка, в которой был начат хотя бы один вызов, навсегда исключалась из продолжения. Так сохраняется различие между запланированной совокупностью протокола и наблюдаемыми строками и предотвращается замена после неудачного ответа, зависящая от исхода.

4.2 Нативные контроли и пропуски

Эталонные контроли объединяют предоставленный контекст кода и каноническое решение. Некорректные контроли добавляют функцию, возвращающую None. Задача допустима для оценки качества только тогда, когда эталонный контроль проходит, а некорректный — не проходит. Нативное оценивание выполняется в автономном контейнере с двумя процессорами, ограничением памяти 3 ГБ и зафиксированными зависимостями BigCodeBench. Для допустимых задач нативный успех кодируется единицей, а нативный сбой или превышение времени теста — нулем. Отсутствующая генерация, незавершенный запуск оценщика и некорректный контроль остаются недоступными. Все 15 000 назначенных строк, включая 15 задач, недопустимых по контролям, и незавершенных кандидатов, сохраняются в учете завершенности и ресурсов.

Генерация дала 14 961 полный кандидат; 39 отправленных назначений остались незавершенными. Каждый завершенный кандидат поступил на нативное оценивание. После эталонного фильтра для анализа качества остаются допустимыми 985 задач, а 264 назначенные конечные точки недоступны из-за состояния генерации или контроля. Неизвестные исходы не подставляются как сбои или успехи. Где это полезно, сообщаются границы с назначенным знаменателем, показывающие экстремальное влияние отсутствующих бинарных исходов; они не являются доверительными интервалами и не исправляют смещение отбора.

Сохраненная классификация конечных точек отделяет сбой генерации от сбоя тестирования. Для D, SR, SN, MR и MN соответственно имеется 4, 3, 2, 13 и 17 незавершенных генераций и 9, 6, 9, 7 и 7 превышений времени нативных тестов. Ни один завершенный основной ответ не нарушает зафиксированное правило извлечения одного блока. Обычные нативные сбои объясняют остальные неуспешные оцененные программы. Записи показывают место сбоя, но дизайн не содержит контрфактической трассы исправления, поэтому нельзя приписать сбой трехшагового процесса конкретному промежуточному сообщению.

Контрольный фильтр подтверждает, что каноническое решение выполняется, а намеренно неверный кандидат дает сбой до интерпретации результатов модели. Задача, не прошедшая любой из контролей, остается в сводках назначений и ресурсов, но исключается из оцениваемой величины качества. Это не позволяет переклассифицировать недоступный или нечувствительный тест после просмотра ответов. Правило извлечения столь же явно: завершенный ответ должен содержать ровно один распознанный огражденный блок Python. Отсутствующий блок, несколько блоков или другая языковая метка дают пустой наблюдаемый кандидат и флаг формата, тогда как незавершившийся вызов остается недоступным. Поэтому некорректно оформленные ответы вносят измеренную стоимость генерации, а отсутствующие ответы могут иметь только частичные или неизвестные счетчики.

Оценщик запускает каждого кандидата на неизменных исходных тестах в зафиксированном локальном окружении. Превышения времени вызова модели и оценщика записываются отдельно; тестируется только итоговый кандидат. Нативный успех, обычный сбой и превышение времени нативного теста сохраняются как разные записи. Сгенерированное

моделью объяснение, напечатанный пример или внутренняя проверка не могут заменить исполнимый успех.

4.3 Использование систем ИИ в исследовании

Оцениваемые программы являются данными, сгенерированными ИИ, поэтому использование модели относится к методу, а не только к помощи в написании текста. GPT-5.6 Luna со средним уровнем рассуждения получила 15 000 назначений основной генерации 11–13 сентября 2026 года и 6 000 выбранных процессов сравнения SCC 13 сентября через Codex CLI/runtime 0.153.4. Из них завершились 14 961 основная генерация и 5 595 процессов сравнения SCC; незавершенные попытки сохраняются в журналах назначений. GPT-5.4-mini сгенерировала более раннее подтверждающее исследование 200 задач 8 сентября 2026 года [17]. Зафиксированные записи сохраняют точные промпты, псевдонимы моделей, порядок запросов, число байтов контекста, временные метки, события состояния, результаты извлечения, счетчики токенов и команды оценщика. Seed выборки поставщика и неизменяемые идентификаторы весов были недоступны; неподтвержденные значения для них не выводятся. Во время основной генерации модели не имели инструментов, браузера, доступа к репозиторию, тестов бенчмарка, эталонных решений или обратной связи оценщика.

OpenAI Codex с GPT-6 помогал в подготовке протокола, реализации эксперимента, коде анализа и упаковки, проверках согласованности, поиске литературы и языковой правке 8–20 сентября 2026 года и при итоговом аудите 30 сентября. Сервис использовался через настольный интерфейс и командную строку и не предоставлял неизменяемый идентификатор ревизии весов. Он создал черновой текст протокола и части исполняемого кода. Проверка включала модульные и интеграционные тесты, повторение численного анализа по зафиксированным журналам, сопоставление таблиц с исходными сводками, сверку цитируемых утверждений с первичными записями и визуальный просмотр собранного PDF. Эти проверки проводились в процессе работы с помощью ИИ; они не подтверждают независимую проверку человеком каждого изменения. Сообщаемые численные результаты получены из сохраненных машиночитаемых записей версионизируемыми скриптами и проверены предоставленными процедурами воспроизведения. Текст модели не рассматривается как доказательство. Зафиксированные протоколы и поправки отражают решения о дизайне и время их принятия. Авторы несут ответственность за рукопись. Рецензии с помощью ИИ служили внутренним контролем качества и не представлены как внешнее научное рецензирование.

5 Результаты

5.1 Качество в исследовании 1000 задач

Наблюдаемые доли успеха в пяти условиях лежат от 50,07% до 54,59%. Контраст рольевых и нейтральных меток равен $-0,75$ процентного пункта при одном вызове (95%-й интервал $[-1,97, 0,44]$) и $+1,35$ пункта при трех вызовах (95%-й интервал $[0,17, 2,56]$). Описательный интервал для трехшагового контраста не включает ноль, однако скорректированное по Холму значение $p = 0,0599$ превышает заранее заданный семейный порог. В основном анализе ни одна нулевая гипотеза о рольевых метках не отвергается.

Контрасты структуры вызовов крупнее и направлены одинаково. На полных допустимых парах задач три вызова снижают успешность на 4,17 пункта при нейтральных инструкциях и на 2,13 пункта при рольевых инструкциях; оба теста отвергают свои нулевые гипотезы после поправки Холма. Обе границы для всех назначений остаются ниже нуля, даже если

Таблица 2. Факторное исследование Luna: число назначений и наблюдений.

Условие	Назначено	Сгенерировано	Наблюдается	Успехи	Успех (%)
D	3000	2996	2951	1611	54.59
SN	3000	2998	2953	1604	54.32
SR	3000	2997	2952	1581	53.56
MN	3000	2983	2938	1471	50.07
MR	3000	2987	2942	1514	51.46

«Наблюдается» означает допустимые конечные точки качества при нативной проверке, а не отдельные задачи. Доля успеха равна числу успешных исходов, деленному на число наблюдаемых повторов. В каждом условии на задачу назначено три повтора.

Таблица 3. Факторные контрасты качества Luna. Разности и границы указаны в процентных пунктах.

Контраст	Задачи	Разность [95% интервал]	p Холма	Границы по назначениям
SR – SN	982	–0.75 [–1.97, 0.44]	0.2357	[–2.33, 0.83]
MR – MN	964	+1.35 [0.17, 2.56]	0.0599	[–0.63, 3.37]
MN – SN	968	–4.17 [–5.65, –2.75]	1.34×10^{-7}	[–6.00, –2.37]
MR – SR	971	–2.13 [–3.43, –0.82]	0.0044	[–3.83, –0.30]

Для задачи необходимы три наблюдаемых повтора в каждом из двух сравниваемых условий. Границы по назначениям используют все 1 000 задач и каждый назначенный повтор; они не являются доверительными интервалами. Интервалы по группам исходных задач для четырех строк равны [–1.99, 0.47], [0.14, 2.60], [–5.69, –2.72] и [–3.44, –0.79].

отсутствующим конечным точкам присвоить значения, наиболее благоприятные для трехшагового условия. Описательное взаимодействие роли и числа вызовов равно +2,08 пункта с интервалом [0.42, 3.74] на общем подмножестве из 963 задач; этот разведочный анализ не входил в семейство четырех тестов. Это исходы отдельных генераций, а не результат выбора лучшего из трех.

5.2 Прямая базовая линия и аудиты истории выборки

Оцененные разности прямого решателя и одношаговых процессов малы: D–SN равна +0,27 пункта, а D–SR — +0,95 пункта; оба интервала пересекают ноль (табл. 4). Эти сравнения не устанавливают эквивалентность. На наблюдаемых парах задач доля успеха D на 3,20–4,59 процентного пункта выше, чем у трехшаговых процессов. Поскольку прямые контрасты не входили в четыре заранее заданных факторных теста, мы приводим их как разведочные оценки и не расширяем подтверждающее семейство после просмотра результатов.

Оценки топологии также отрицательны в обеих подвыборках, определенных историей выборки (табл. 5). Эффект трех нейтральных вызовов равен –4,17 пункта и для 200 повторно использованных идентификаторов, и для 800 новых. Контраст топологии с ролевыми метками отрицателен в обеих подвыборках, но имеет большую неопределенность в меньшей повторно использованной части. Контрасты ролевых меток остаются малыми и неопределенными. Этот анализ не исключает загрязнение бенчмарка, однако показывает, что основной результат для топологии не определяется только 200 ранее исследованными идентификаторами задач.

Таблица 4. Разведочные контрасты качества прямого решателя. Разности и интервалы указаны в процентных пунктах; положительные значения благоприятствуют D.

Контраст	Задачи	Разность	95% интервал
D – SN	980	+0.27	[−0.88, 1.43]
D – SR	979	+0.95	[−0.17, 2.11]
D – MN	966	+4.59	[2.97, 6.18]
D – MR	970	+3.20	[1.82, 4.64]

Интервалы получены повторной выборкой задач. Эти апостериорные контрасты не входят в семейство из четырех тестов с поправкой Холма и предназначены для оценивания, а не для образования нового подтверждающего семейства.

Таблица 5. Чувствительность четырех основных контрастов к старым и новым задачам. Значения указаны в процентных пунктах с 95%-ми интервалами бутстрепа по задачам.

Контраст	Подмножество	Задачи	Разность [95% интервал]
SR–SN	повторные 200	194	−1.03 [−3.78, 1.72]
	новые 800	788	−0.68 [−2.07, 0.72]
MR–MN	повторные 200	190	+1.93 [−0.70, 4.56]
	новые 800	774	+1.21 [−0.13, 2.58]
MN–SN	повторные 200	192	−4.17 [−7.64, −0.87]
	новые 800	776	−4.17 [−5.84, −2.58]
MR–SR	повторные 200	190	−1.23 [−3.68, 1.23]
	новые 800	781	−2.35 [−3.88, −0.81]

Каждый ответ был сгенерирован заново. «Повторные» означает, что идентификатор задачи встречался в прежнем исследовании 200 задач; сам результат повторно не использовался.

5.3 Учет ресурсов

Для завершенных кандидатов учет ресурсов включает весь повторно переданный ввод и раскрытые промежуточные ответы. Одношаговые инструкции с ролевыми метками имеют отношение стоимостных оценок 0,939 к нейтральным инструкциям, то есть на 6,08% меньшую API-эквивалентную оценку, на 997 задачах со всеми тремя наблюдаемыми повторами в обоих условиях; отношение токенов равно 0,986. Внутри трехшагового режима соответствующее отношение стоимости равно 0,978, а токенов — 0,990 на 979 парах задач. Эти связанные с метками различия ресурсов являются описательными и меньше наблюдавшихся в более раннем мини-исследовании.

Трехшаговые нейтральные инструкции используют в 3,08 раза больше токенов и имеют в 2,67 раза большую стоимостную оценку, чем одношаговые нейтральные инструкции. Для ролевых меток отношения равны 3,09 и 2,78. После удаления скидки на кеш отношения стоимости составляют 2,78 и 2,87. Денежные величины являются API-эквивалентными оценками по измеренным счетчикам, а не счетами за подписку или полной стоимостью исследования. Токены рассуждения включены в вывод и не добавляются повторно.

Для масштаба средние известные итоги на завершенного кандидата равны 3 972 токенам и \$0,000813 для D, 4 073 и \$0,000899 для SR, 4 131 и \$0,000957 для SN, 12 583 и \$0,002496 для MR, а также 12 712 и \$0,002552 для MN. Совокупная известная стоимостная оценка пяти ветвей равна \$23,07. Основным сравнением ресурсов остаются отношения, поскольку небольшие различия числа завершений делают непарные суммы вводящими в заблуждение.

Таблица 6. Отношения ресурсов в факторном исследовании Luna: первое условие делится на второе.

Контраст	Задачи	Токены	Стоимость	95% интервал	Без кеша
SR – SN	997	0.986	0.939	[0.93, 0.95]	0.955
MR – MN	979	0.990	0.978	[0.97, 0.99]	0.986
MN – SN	983	3.078	2.672	[2.63, 2.71]	2.777
MR – SR	986	3.090	2.780	[2.74, 2.82]	2.867

Отношения делят суммы трехповторных средних по задачам на полных ресурсных парах. Показанный интервал относится к отношению API-эквивалентных стоимостных оценок. Вариант «Без кеша» устраняет скидку на кешированный ввод; абонентская плата не оценивается.

Таблица 7. Более ранняя подтверждающая проверка GPT-5.4-mini на 200 повторно использованных идентификаторах задач. Разности и описательные интервалы указаны в процентных пунктах; n — число полных пар задач. Четыре точных теста Макнемара образуют семейство с поправкой Холма.

Контраст	n	Δ	95% интервал	p Холма
SR-SN	191	−1.05	[−4.71, +2.62]	1.000
MR-MN	193	+1.04	[−3.11, +5.70]	1.000
MN-SN	191	−4.71	[−9.42, 0.00]	.313
MR-SR	193	−2.07	[−6.22, +2.07]	1.000

Для числа входных токенов I , кешированной части C и числа выходных токенов O оценка Luna равна $[0.20(I - C) + 0.02C + 1.20O]/10^6$ долларов по раскрытым базовым справочным тарифам сентября 2026 года [15, 18]. Она оценивает наблюдаемые счетчики по опубликованному API-тарифу и не включает накладные расходы поставщика, время исследователя, вычисления оценщика, доплаты за запись в кеш или полную стоимость владения. Анализ без кеша использует $[0.20I + 1.20O]/10^6$ и проверяет, зависит ли направление результата от скидки на кешированный ввод. Каждый переданный контекст и раскрытый предыдущий ответ включены, поскольку такое раскрытие является частью воздействия. Неизвестные счетчики отмечаются как неизвестные и не заменяются нулями.

5.4 Подтверждающие исследования

Более раннее факторное исследование GPT-5.4-mini использовало те же 200 идентификаторов задач, позднее сохраненных в расширенном распределении, но создавало по одному независимому запросу на условие с прежней моделью. Из 1 000 назначений завершились 996; контрасты включают 191–193 допустимые пары задач. Все четыре оцененных эффекта имеют то же направление, что и в исследовании Luna (табл. 7): эффекты ролевых меток отрицательны при одном вызове и положительны при трех, тогда как эффекты структуры вызовов отрицательны при обоих вариантах меток. После поправки Холма ни один из четырех тестов не отвергает нулевую гипотезу. Подтверждение относится к совпадению направления для двух псевдонимов моделей на тех же 200 задачах, а не к независимой выборке задач или второму мощному подтверждению. В мини-исследовании отношения стоимостной оценки трех вызовов равны 2,09 относительно SN и 2,50 относительно SR.

Другие сохраненные пилотные эксперименты варьируют размещение инструкций или меняют процедуру рассуждения. Мы не объединяем их с основным исследованием.

Таблица 8. Качество SCC на наблюдаемых сопоставленных повторах. Разности и интервалы указаны в процентных пунктах; n — число участвующих задач.

Контраст	n	Δ	95% ДИ	p Холма
SCC–SR	883	–2.68	[–4.72, –0.60]	.0217
SCC–SN	882	–2.48	[–4.50, –0.43]	.0217

6 Сравнение с Self-Collaboration

Мы сравниваем Self-Collaboration (SCC) с одношаговыми условиями, используя ту же модель Luna, средний уровень рассуждения и нативный оценщик. Это сравнение относится к полным процессам. Адаптированный авторский контроллер [4] начинает с JSON-плана аналитика, вызывает разработчика для получения Python-кода, а затем просит тестирующего сгенерировать до пяти примеров вызова в функции `check(candidate)`. Он выполняет программу и сгенерированную проверку; внутренним сигналом остановки служит выполнение без исключений, при этом напечатанные значения автоматически не сравниваются с ожидаемыми ответами. Заданный предел допускает одну первоначальную реализацию и не более одного исправления. Модель получает полный предоставленный контекст задачи и сериализованную историю ролей. Эталонные решения и тесты бенчмарка остаются внешними.

Первоначальный план содержит 3 000 блоков «задача–повтор», в каждом из которых назначаются новые процессы SR, SN и SCC. Поправка, принятая после начала генерации, но до проверки качества SCC, фиксирует первые 2 000 рандомизированных блоков (seed порядка 20260911). Это дает 6 000 назначений методов на 956 задачах: у 212 задач выбран один повтор, у 444 — два, у 300 — три. Из этих задач 941 проходит эталонные проверки. Невыбранные 3 000 назначений методов не входят в знаменатель после поправки.

Нативное оценивание прошли все 5 595 завершенных программ: 1 997 SR, 1 998 SN и 1 600 SCC. В сравнении сохранены 397 инфраструктурных сбоев и восемь ранее приостановленных попыток. В частности, у SCC имеется 396 сбоев Docker и четыре приостановленных процесса; у SR — три приостановленные попытки; у SN — одна приостановленная попытка и один сбой потока. Ни одна отправленная попытка не генерировалась повторно. После эталонных проверок качество наблюдается для 1 964 повторов SR, 1 965 повторов SN и 1 574 повторов SCC; число успехов равно соответственно 1 067, 1 064 и 809. Это количества наблюдаемых повторов, а не независимых задач и не показатели выбора лучшего из трех.

Для каждого компаратора мы сначала усредняем разности исходов SCC и компаратора внутри каждой задачи по совпадающим выбранным повторам с обоими наблюдаемыми исходами. Затем усредняем эти средние с одинаковым весом по допустимым задачам, имеющим хотя бы один сопоставленный повтор. Двусторонние t -тесты на уровне задач для SCC–SR и SCC–SN образуют отдельное семейство из двух тестов с поправкой Холма. Интервалы используют 10 000 выборок целых задач, генератор PCG64, seed 20260911 и линейные процентильные границы.

Оба условных контраста отрицательны (табл. 8). Интервалы чувствительности по группам исходных задач сходны: [–4.72, –0.58] пункта относительно SR и [–4.51, –0.43] относительно SN для объединенного разбиения. Однако экстремальные границы с фиксированным знаменателем по всем 941 допустимым выбранным задачам равны [–12.98, +6.61] и [–13.14, +6.43] пункта. Они допускают любой знак. Поэтому результаты по наблюдаемым парам не устанавливают худшее качество SCC для всей назначенной совокупности.

Для сравнения ресурсов необходимы оба полностью завершённых процесса с корректными счетчиками. Отношение парных средних по задачам для API-эквивалентной стоимостной оценки равно 3,138 относительно SR (896 задач; 95%-й интервал [3.067, 3.208]) и 2,939 относительно SN (895 задач; [2.871, 3.010]). Эти интервалы отношений средних используют 10 000 выборок целых задач с seed 20260914. Они относятся к наблюдаемым полным процессам и не объединяют сохранение качества с денежным преимуществом. Доступность и сопоставленные эффекты до и после поправки сохранены как описательные проверки; они не идентифицируют причинный временной эффект.

Отдельная диагностика оценивает все 399 сохранённых программ разработчика из незавершённых процессов SCC, не перезапуская контроллер и не добавляя сгенерированные им проверки. Среди 392 программ, допустимых по контролям, она находит 196 успешных; семь программ не проходят контроль допустимости. Эти результаты промежуточных программ не заменяют неизвестные исходы полного процесса в сравнении.

7 Обсуждение

Завершённое факторное исследование оценивает условную разность между проверенными процессами. В протоколе полного контекста без сжатия распределение планирования, реализации и критики между тремя отдельными вызовами увеличивает повторную передачу и измеренный вывод и даёт меньшую успешность нативных тестов на наблюдаемых парах задач, чем размещение тех же операций в одном ответе. Результат относится к исследованной структуре вызовов, модели, совокупности задач, пределу времени и контракту оценивания. Он не показывает, что отдельные агенты в общем случае неэффективны или что дополнительное взаимодействие не может помочь, когда у агентов есть инструменты, поиск, состояние репозитория или выровненный вычислительный бюджет.

Вмешательство ролевых меток меньше и неопределённое. Оценка роли при трех вызовах описательно положительна, но не достигает скорректированного по Холму порога значимости в основном анализе. Оценки неточны и имеют разные знаки при разных структурах вызовов, поэтому не устанавливают преимущества от именованного этапа промпта в этом режиме. Неотвержение также не устанавливает эквивалентность качества. Внутренние метки все же могут организовать промпт или передать полезную декомпозицию; эксперимент измеряет, изменили ли они исход бенчмарка при одинаковых формулировках операций.

Использование ресурсов следует интерпретировать вместе с качеством. Одношаговые ролевые метки немного снижают измеренную стоимостную оценку в этом исследовании, тогда как три вызова стоят примерно в 2,7–2,8 раза дороже. Поскольку дизайн не задаёт строгий выровненный бюджет вывода, учёт включает обусловленную длину рассуждения, повторный контекст, промежуточные ответы и состояние кеша. Заявление о совместном превосходстве по качеству и стоимости не делается. Анализ без кеша изменяет величину, но сохраняет большой контраст топологии.

Внешний оценщик также не позволяет процессу получить зачет за убедительное самоописание исправления, которое не изменило исполнимое поведение. В нативные тесты поступает только итоговый кандидат. У прямого условия наименьшие наблюдаемые средние ресурсы. Его оцененные различия качества с одношаговыми поэтапными условиями малы, а интервалы включают ноль; эти разведочные тесты не предназначены для установления эквивалентности. На наблюдаемых парах задач прямое условие успешнее обоих трехшаговых условий, хотя эти сравнения не были заранее зарегистрированы как подтверждающее семейство.

В рассматриваемом режиме прямой и одношаговый поэтапный процессы являются менее дорогими базовыми линиями. Дополнительные вызовы следует оценивать, когда

они добавляют параллельный поиск, закрытое состояние, инструменты, извлечение данных или исполнимую обратную связь. При оценивании следует сообщать и парное по задачам исполнимое качество, и не зависящее от поставщика число токенов.

Три повтора улучшают оценку для выбранных задач, но не превращают 15 000 назначений в независимые единицы-задачи. Бутстреп сохраняет повторы вместе, а каждая задача вносит среднюю успешность на генерацию с одинаковым весом. Это не позволяет считать повторные запросы новыми единицами-задачами или выбирать лучший результат из трех.

Вспомогательные исследования по одному изменяют модель, размещение инструкций или процедуру рассуждения. Их результаты отделены от тестов качества Luna и не устраняют возможность загрязнения обучающими данными бенчмарка.

8 Ограничения и воспроизводимость

Работа использует два псевдонима моделей, каждый с одним уровнем рассуждения и одной версией клиента, и не идентифицирует эффекты семейства моделей. Меньшее мини-исследование повторно использует те же 200 идентификаторов задач и имеет только один запрос на условие; совпадение направления является подтверждением, а не независимой репликацией. Поставщик не раскрыл неизменяемый идентификатор весов, значение температуры или seed выборки. Расширенная выборка повторно использует эти 200 идентификаторов, поэтому не является 1 000 полностью новыми отложенными задачами. Публичная доступность бенчмарка оставляет загрязнение обучающих данных неразрешенным, а родственные задачи могут уменьшать эффективную независимость.

Качество остается неизвестным для недоступных генераций и инфраструктурных сбоев. Наблюдаемые сбои тестов и превышения времени бенчмарка считаются неуспешными программами на допустимых задачах. После транспортных прерываний продолжение разрешалось только для еще не начатых назначений; начатые вызовы не повторялись. Оценки по доступным случаям относятся к наблюдаемому допустимому подмножеству. Экстремальные границы по назначениям показывают, что допускают отсутствующие бинарные конечные точки, но не устраняют неслучайные пропуски. Поэтому исследование избегает заявлений об эквивалентности, не меньшей эффективности или экономически оптимальном дизайне.

Нативные тесты дают исполнимое свидетельство, но наследуют связь между спецификациями и тестами бенчмарка. Эталонный и некорректный контроли выявляют непригодные окружения и нечувствительные негативные проверки; они не доказывают, что каждое утверждение отражает задачу на естественном языке. Протокол фиксированного контекста исключает поиск, инструменты, взаимодействие с репозиторием и адаптивную обратную связь от тестов. Следовательно, результаты не устанавливают качество исправления репозитория или сквозной разработки программного обеспечения.

Вмешательство уже, чем более широкий прототип, который его мотивировал: выбор контекста, выборочные перезапуски, вспомогательная проверка и сжатие отключены. Это изолирует ядро поэтапной генерации, но не заменяет полную оценку развертывания. Сравнение SCC имеет дополнительное ограничение, поскольку его сгенерированные проверки и сигнал остановки при отсутствии исключений задают другой контроллер обратной связи. Более высокая стоимость и меньшая успешность SCC на наблюдаемых парах задач относятся ко всему процессу и не изолируют вклад ролевых меток внутри SCC. Значительные инфраструктурные пропуски SCC ограничивают выводы на уровне совокупности.

Идентификаторы задач, промпты, ответы, состояния, флаги извлечения, счетчики, контроли, нативные отчеты и seed анализа сохранены в материалах проекта. Эти записи позволяют восстановить знаменатели назначений и парные подмножества. Новая ревизия

поставщика, версия зависимости или интерпретация связи задачи и теста может изменить доли. При воспроизведении следует сохранить указанные псевдоним модели, версию клиента, байты контекста, пределы времени, контрольный фильтр и правила пропусков и сообщать любое отклонение.

9 Заключение

На 15 000 назначенных генерациях Luna после поправки Холма не отвергается ни одна нулевая гипотеза о ролевых метках. Распределение тех же операций с полным контекстом между тремя вызовами снижает парную исполнимую успешность на 2,13–4,17 процентного пункта и увеличивает API-эквивалентную стоимостную оценку в 2,67–2,78 раза. Отрицательное направление обоих контрастов топологии также наблюдается в меньшем мини-исследовании и сохраняется для 800 новых идентификаторов задач Luna. Прямой решатель имеет малые и неопределенные различия с одношаговыми поэтапными процессами и в разведочных сравнениях показывает большую успешность на наблюдаемых парах задач, чем оба трехшаговых процесса. Сравнение 2 000 блоков SCC также показывает большую стоимость и меньшую успешность на наблюдаемых парах задач, чем новые одношаговые контроли, однако отсутствующие исходы не позволяют ранжировать качество для всей совокупности. Эксперименты разделяют два решения: сами названия ролей не дали надежного выигрыша, тогда как дополнительные вызовы с полным контекстом увеличили стоимость и снизили парную успешность.

Доступность данных

К рукописи приложен анонимизированный пакет воспроизводимости. Он содержит зафиксированные протоколы и поправки, выборки задач, исходы на уровне назначений, счетчики состояния и использования, основные программы-кандидаты, все основные отчеты нативной проверки, свидетельства контролей, спецификации зафиксированного окружения, исходный код и данные оценщика бенчмарка под их первоначальной лицензией, разбиения по зависимости исходных задач, зафиксированные исходники контроллера и транспорта SCC с промптами и лицензией исходной реализации, скрипты анализа, точные входы таблиц и манифест хешей файлов. Пакет позволяет повторно выполнить численные анализы основного, мини- и SCC-исследований и компонентов исходных задач; проверить главные табличные значения; а также подтвердить, что основные программы и нативные состояния точно совпадают с неизменяемыми записями оценщика без учетных данных поставщика. Полные тела диалоговых ответов, исходный текст кандидатов SCC и рабочие транспортные трассы не включены, поскольку не нужны для этих проверок и могут содержать случайные сгенерированные идентификаторы или локальные метаданные машины. Исключены история репозитория, способная раскрыть личности авторов, локальные пути, идентификаторы учетных записей и учетные данные. Полный сохраненный архив будет опубликован после периода двойного анонимного рецензирования с учетом условий бенчмарка и поставщика.

Список литературы

- [1] Miltiadis Allamanis. 2019. The Adverse Effects of Code Duplication in Machine Learning Models of Code. In *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*. ACM, 143–153. doi:10.1145/3359591.3359735
- [2] Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2023. CodeT: Code Generation with Generated Tests. In *The Eleventh International Conference on Learning Representations*. <https://openreview.net/forum?id=ktw68Cmu9c>

- [3] Junhao Chen, Xiang Li, Yibin Xu, Yuehan Cui, Fangsheng Weng, Hao Zhao, Fei Ma, and Qi Tian. 2026. PairCoder: Pair Programming-Inspired Two-Agent Collaboration for Code Generation. In *Findings of the Association for Computational Linguistics: ACL 2026*. Association for Computational Linguistics, 3043–3058. doi:10.18653/v1/2026.findings-acl.149
- [4] Yihong Dong. 2024. *Self-Collaboration Code Generation: Accompanying Python Software*. Historical author implementation associated with arXiv:2304.07590, pinned at commit b471e12051190dbae2c71b429a3c87466df4b336; workflow and transport adaptations are specified in the text.. <https://github.com/YihongDong/Self-collaboration-Code-Generation>
- [5] Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2024. Self-Collaboration Code Generation via ChatGPT. *ACM Transactions on Software Engineering and Methodology* 33, 7 (2024), 1–38. Preprint: arXiv:2304.07590. doi:10.1145/3672459
- [6] Junda He, Christoph Treude, and David Lo. 2025. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Transactions on Software Engineering and Methodology* 34, 5 (2025), 1–30. doi:10.1145/3712003
- [7] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=VtmBAGCN7o>
- [8] Yunsong Hong. 2026. Learning How Much to Collaborate: Difficulty-Aware Topology Selection for Multi-Agent Code Generation. arXiv:2609.13890 <https://arxiv.org/abs/2609.13890>
- [9] Dong Huang, Jie M. Zhang, Michael Luck, Qingwen Bu, Yuhao Qing, and Heming Cui. 2023. AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation. arXiv:2312.13010 doi:10.48550/arXiv.2312.13010
- [10] Md. Ashraful Islam, Mohammed Eunus Ali, and Md Rizwan Parvez. 2024. MapCoder: Multi-Agent Code Generation for Competitive Problem Solving. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 4912–4944. doi:10.18653/v1/2024.acl-long.269
- [11] Sayash Kapoor, Benedikt Stroebel, Zachary S. Siegel, Nitya Nadgir, and Arvind Narayanan. 2025. AI Agents That Matter. *Transactions on Machine Learning Research* (2025). <https://openreview.net/forum?id=Zy4uFzMvIZ>
- [12] Pedro Henrique Luz de Araujo, Paul Röttger, Dirk Hovy, and Benjamin Roth. 2025. Principled Personas: Defining and Measuring the Intended Effects of Persona Prompting on Task Performance. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 26857–26886. doi:10.18653/v1/2025.emnlp-main.1364
- [13] James G. MacKinnon, Morten Ørregaard Nielsen, and Matthew D. Webb. 2023. Cluster-Robust Inference: A Guide to Empirical Practice. *Journal of Econometrics* 232, 2 (2023), 272–299. doi:10.1016/j.jeconom.2022.04.001
- [14] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-2019
- [15] OpenAI. 2026. *API Pricing*. Standard text-token rates. <https://developers.openai.com/api/docs/pricing>
- [16] OpenAI. 2026. *Codex Non-Interactive Mode and JSONL Event Stream*. <https://developers.openai.com/codex/noninteractive>
- [17] OpenAI. 2026. *GPT-5.4 Mini Model Documentation*. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>
- [18] OpenAI. 2026. *GPT-5.6 Luna Model Documentation and Pricing*. <https://developers.openai.com/api/docs/models/gpt-5.6-luna>
- [19] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 15174–15186. doi:10.18653/v1/2024.acl-long.810
- [20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, Vol. 36. doi:10.52202/075280-0377
- [21] Haoran Sun and Shaoning Zeng. 2025. Introspection of Thought Helps AI Agents. arXiv:2507.08664 <https://arxiv.org/abs/2507.08664>
- [22] Dat Tran and Douwe Kiela. 2026. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460 <https://arxiv.org/abs/2604.02460>
- [23] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2025. Demystifying LLM-Based Software Engineering Agents. *Proceedings of the ACM on Software Engineering* 2, FSE (2025), 801–824. doi:10.1145/3715754
- [24] Jiawei Xu, Arief Koesdwiady, Sisong Bei, Yan Han, Baixiang Huang, Dakuo Wang, Yutong Chen, Zheshen Wang, Peihao Wang, Pan Li, and Ying Ding. 2026. Rethinking the Value of Multi-Agent Workflow: A Strong Single Agent Baseline. arXiv:2601.12307 <https://arxiv.org/abs/2601.12307>

- [25] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. Published as a NeurIPS 2024 paper; arXiv version retained for stable metadata. arXiv:2405.15793 <https://arxiv.org/abs/2405.15793>
- [26] Huan Zhang, Wei Cheng, Yuhao Wu, and Wei Hu. 2024. A Pair Programming Framework for Code Generation via Multi-Plan Exploration and Feedback-Driven Refinement. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ACM, 1319–1331. doi:10.1145/3691620.3695506
- [27] Mingqian Zheng, Jiaxin Pei, Lajanugen Logeswaran, Moontae Lee, and David Jurgens. 2024. When “A Helpful Assistant” Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 15126–15154. doi:10.18653/v1/2024.findings-emnlp.888
- [28] Li Zhong, Zilong Wang, and Jingbo Shang. 2024. Debug like a Human: A Large Language Model Debugger via Verifying Runtime Execution Step-by-Step. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 851–870. doi:10.18653/v1/2024.findings-acl.49
- [29] Yueheng Zhu, Chao Liu, Xuan He, Xiaoxue Ren, Zhongxin Liu, Ruwei Pan, and Hongyu Zhang. 2026. AdaCoder: An Adaptive Planning and Multi-Agent Framework for Function-Level Code Generation. *IEEE Transactions on Software Engineering* 52, 2 (2026), 631–650. doi:10.1109/TSE.2025.3642621
- [30] Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widayarsi, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, Simon Brunner, Chen Gong, Thong Hoang, Arnel Randy Zebaze, Xiaoheng Hong, Wen-Ding Li, Jean Kaddour, Ming Xu, Zhihan Zhang, Prateek Yadav, Naman Jain, Alex Gu, Zhoujun Cheng, Jiawei Liu, Qian Liu, Zijian Wang, David Lo, Binyuan Hui, Niklas Muennighoff, Daniel Fried, Xiaoning Du, Harm de Vries, and Leandro von Werra. 2025. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. In *International Conference on Learning Representations*. Published as a conference paper at ICLR 2025. <https://arxiv.org/abs/2406.15877>